

Desenvolupament web en entorn servidor

PYTHON

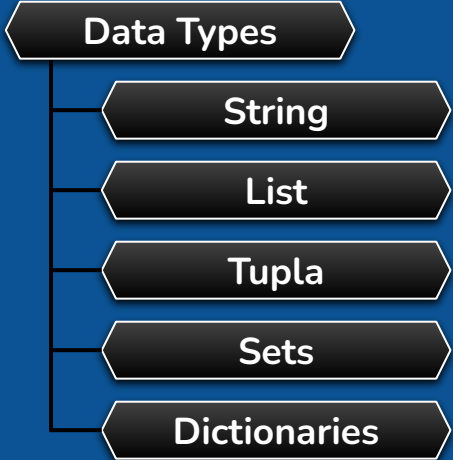
RA's vinculades:

RA3: Escriu blocs de sentències embeguts en llenguatges de marques, seleccionant i utilitzant les estructures de programació.

PYTHON

INDEX

Data Types



```
graph TD; A[Data Types] --- B[String]; A --- C[List]; A --- D[Tupla]; A --- E[Sets]; A --- F[Dictionaries];
```

String

List

Tupla

Sets

Dictionaries

Data Types

String

Diagram illustrating the indexing of a string "ROGER [] S O B R I N O". The indices range from 0 to 12 for positive indexing and -13 to -1 for negative indexing. The word "index" is labeled with arrows pointing to the index values.

Index	Character
0	R
1	O
2	G
3	E
4	R
5	[]
6	S
7	O
8	B
9	R
10	I
11	N
12	O
-13	R
-12	O
-11	G
-10	E
-9	R
-8	[]
-7	S
-6	O
-5	B
-4	R
-3	I
-2	N
-1	O

- Extraire caractères d'un String

```
string.py x
1 name = "Roger Sobrino"
2
3 print(name[2], name[6])
```

Run: string x

/home/roger/IJB/IJB/venv/local/bin/p
g S

Process finished with exit code 0

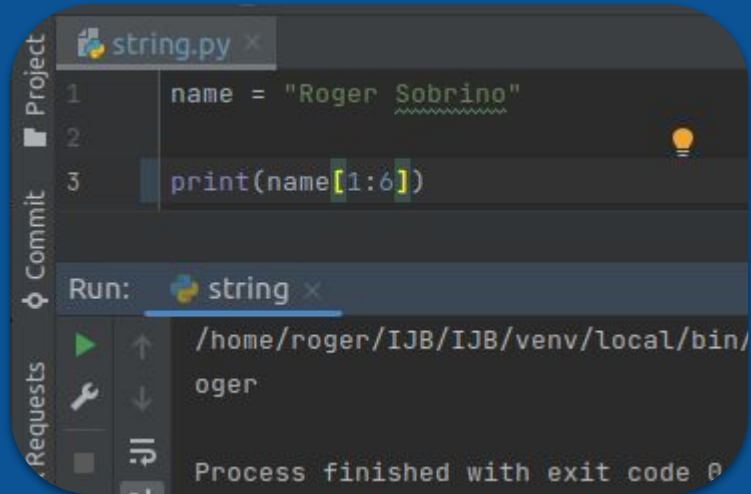
```
string.py x
1 name = "Roger Sobrino"
2
3 print(name[-2], name[-6])
```

Run: string x

/home/roger/IJB/IJB/venv/local/bin/p
n o

Process finished with exit code 0

- Extraire caractères d'un String



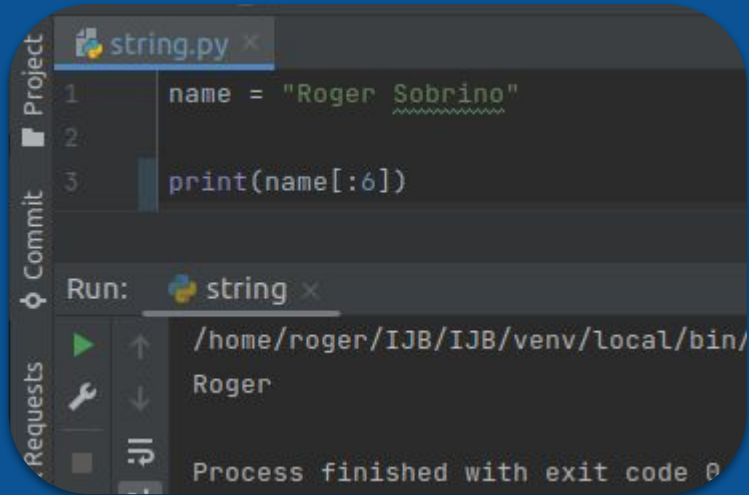
The screenshot shows an IDE with a file named `string.py`. The code contains three lines:

```
1 name = "Roger Sobrino"
2
3 print(name[1:6])
```

Below the code editor, the 'Run' tab is active, showing the command `python string.py` and the output `oger`. The status bar at the bottom indicates 'Process finished with exit code 0'.

0	1	2	3	4	5	6	7	8	9	10	11	12
[R]	[O]	[G]	[E]	[R]	[]	[S]	[O]	[B]	[R]	[I]	[N]	[O]

- Extraire caractères d'un String



The screenshot shows an IDE with a file named `string.py`. The code contains two lines: `name = "Roger Sobrino"` and `print(name[:6])`. The IDE's output console shows the command `Run: string` and the output `Roger`. Below the output, it states "Process finished with exit code 0".

```
1 name = "Roger Sobrino"
2
3 print(name[:6])
```

Run: string

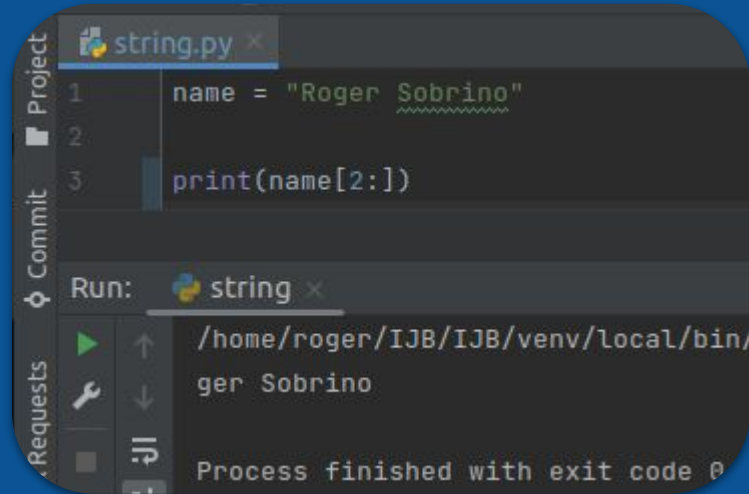
/home/roger/IJB/IJB/venv/local/bin/

Roger

Process finished with exit code 0

0	1	2	3	4	5	6	7	8	9	10	11	12
[R]	[O]	[G]	[E]	[R]	[]	[S]	[O]	[B]	[R]	[I]	[N]	[O]

- Extraire caractères d'un String



The screenshot shows a Python IDE with a file named `string.py`. The code defines a string `name = "Roger Sobrino"` and prints a slice of it: `print(name[2:])`. The output in the console is `ger Sobrino`, demonstrating that the first two characters ('Ro') are excluded. The console also shows the full path to the Python interpreter and the exit code 0.

```
1 name = "Roger Sobrino"
2
3 print(name[2:])
```

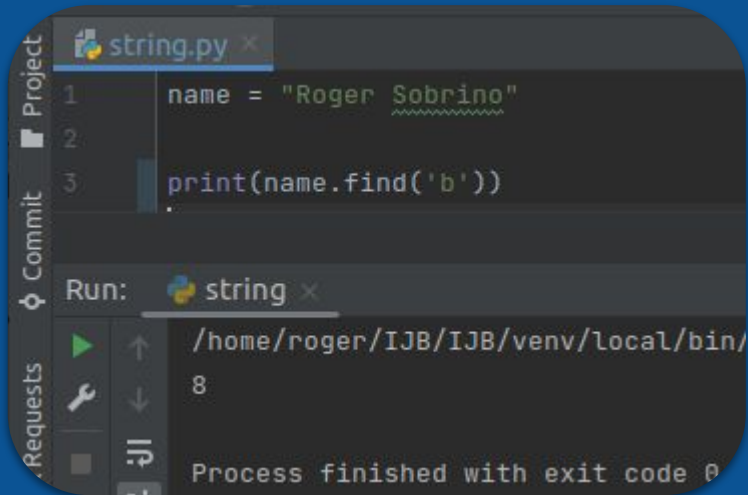
Run: string x

/home/roger/IJB/IJB/venv/local/bin/
ger Sobrino

Process finished with exit code 0

0	1	2	3	4	5	6	7	8	9	10	11	12
[R]	[O]	[G]	[E]	[R]	[]	[S]	[O]	[B]	[R]	[I]	[N]	[O]

- Trobar lletres dintre d'un String amb **find()**.



```
1 name = "Roger Sobrino"
2
3 print(name.find('b'))
```

Run: string x

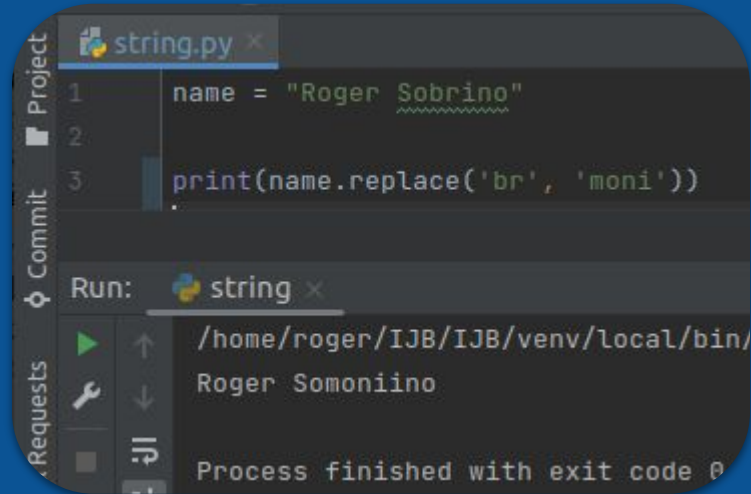
/home/roger/IJB/IJB/venv/local/bin/

8

Process finished with exit code 0

0	1	2	3	4	5	6	7	8	9	10	11	12
[R]	[O]	[G]	[E]	[R]	[]	[S]	[O]	[B]	[R]	[I]	[N]	[O]

- Canviar text d'un String amb **replace()**



The screenshot shows a code editor with a file named `string.py`. The code contains two lines:

```
1 name = "Roger Sobrino"  
3 print(name.replace('br', 'moni'))
```

Below the code editor, the output of the script is displayed:

```
Run: string x  
/home/roger/IJB/IJB/venv/local/bin/  
Roger Somoniino  
Process finished with exit code 0
```

... i més mètodes ⇒

- Son col·leccions d'objectes de qualsevol tipus ordenades de forma arbitraria i sense tamany fixe.
- Son mutables (es poden modificar).
- Son una seqüència de valors indexats per números enters.

```
myList = ['Pablo', 'Juana', 'Maria', 123, 3.34]
```


- Concatenar una o més lists amb `+`.

```
#Concatenació
list1 = [1, 2, 3]
list2 = [4, 4, 6]

print(list1 + list2)
print(list2 + list1)
```

lists x

```
/home/roger/IJB/IJB/venv/local/bin/
[1, 2, 3, 4, 4, 6]
[4, 4, 6, 1, 2, 3]

Process finished with exit code 0
```

- Mirar si un valor està dintre d'una list (retorna un boolean) amb **in**.

```
Iteration
list1 = [3, 9, 0]

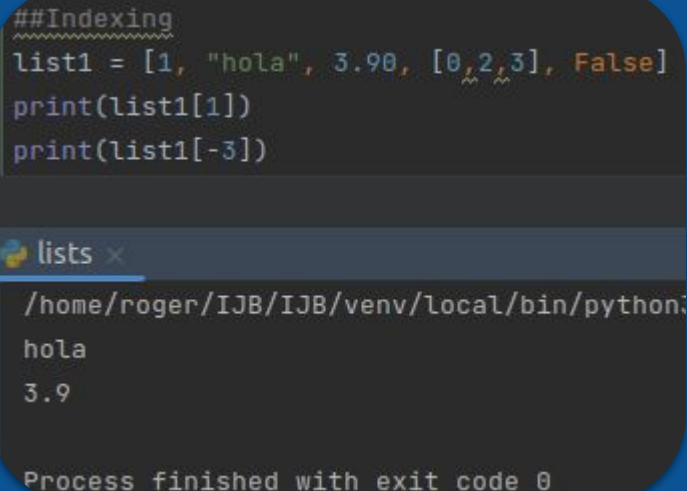
print(3 in list1)
print(6 in list1)

lists x
/home/roger/IJB/IJB/venv/local/bin/p
True
False

Process finished with exit code 0
```

- Quan volem veure un valor concret dintre d'una list ubicat en una posició (index) de la list (indexació).

```
##Indexing
~~~~~
list1 = [1, "hola", 3.90, [0,2,3], False]
print(list1[1])
print(list1[-3])
```



lists x

```
/home/roger/IJB/IJB/venv/local/bin/python:
hola
3.9

Process finished with exit code 0
```

- Les matrius son arrays multidimensionals. En aquest exemple tenim una **matriu 3x3** de lists.

```
matriu1 = [[1,2,3,"hola"],[3,True,0],["Roger", 18, False]]  
  
print(matriu1[1])  
print(matriu1[0][2])  
print(matriu1[0][3])
```

lists x

```
/home/roger/IJB/IJB/venv/local/bin/python3.10 /home/roger/IJB.  
[3, True, 0]  
hola  
Traceback (most recent call last):  
  File "/home/roger/IJB/IJB/M7/Teoria/PYTHON/basics/lists.py"  
    print(matriu1[0][3])  
IndexError: list index out of range
```

- Dintre d'una list es poden modificar els seus valors.

```
#Changing Lists
list1 = ["Roger", "Apple", "Tool"]
list1[2] = 'Funciona'
print(list1)
```

lists x

/home/roger/IJB/IJB/venv/local/bin/
['Roger', 'Apple', 'Funciona']

Process finished with exit code 0

- El mètode **append()** afegeix l'objecte list al final de la list
- El mètode **extend()** inserta els valors al final de la list.

```
Changing Lists
list1 = ["Roger", "Apple", "Tool"]
list1[2] = 'Funciona'

list1.append([3,4])
print(list1)
list1.extend([5,6])
print(list1)
```

lists x

```
/home/roger/IJB/IJB/venv/local/bin/python3.10
['Roger', 'Apple', 'Funciona', [3, 4]]
['Roger', 'Apple', 'Funciona', [3, 4], 5, 6]
```

Process finished with exit code 0

- El mètode **sort()** serveix per ordenar de forma ascendent els valors de la list.

```
sort()
list1 = ["Roger", "Andreu", "Juan", "John", "Doe"]
list2 = [2, 5, 6, 7, 1, 0, 3, 4]
list1.sort()
list2.sort()
print("sort de list1: ")
print(list1)
print("sort de list2: ")
print(list2)
```

lists x

```
/home/roger/IJB/IJB/venv/local/bin/python3.10 /hom
sort de list1:
['Andreu', 'Doe', 'John', 'Juan', 'Roger']
sort de list2:
[0, 1, 2, 3, 4, 5, 6, 7]
```

process finished with exit code 0

- Elimina, de la list, l'últim valor amb el mètode **pop()**.

```
myList = [1, 2, 3, 4]
print(myList.pop())
print(myList)
```



```
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/lists.py
4
[1, 2, 3]
```

Manera diferent de crear una list:

Codi:

```
numbers = [value**2 for value in range(1,11)]  
print(numbers)
```

Resposta:

```
/home/roger/IJB/IJB/venv/bin/python /home,  
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

- No Son mutables (no es poden modificar).
- Son una seqüència de valors indexats per números enters.
- Inicialització Tupla $\Rightarrow$ **myTupla = ()**

```
myTupla = (1, 2, 3, 4, 5, 6, 7)
```

- Mètode **len()** per conèixer la longitud de la Tupla.

```
myTupla = (1, 2, 3, 4, 5, 6, 7)
print(len(myTupla))
```

tupla x

```
/home/roger/IJB/IJB/venv/bin/python
```

```
7
```

```
Process finished with exit code 0
```

- Per buscar un valor dintre d'una tupla s'utilitza **in**.

```
x = 9
y = 3
myTupla = (1,2,3,4,5,6,7)
print(x in myTupla)
print(y in myTupla)
```

```
tupla x
/home/roger/IJB/IJB/venv/bin/python /t
False
True
Process finished with exit code 0
```

- Per saber quantes vegades es repeteix un valor dintre d'una tupla es fa servir el mètode **count()**.

```
myTupla = (1, 2, "pedro", 3, 4, "pedro", 5, 6, 7)
print(myTupla.count("pedro"))
```

tupla x

```
/home/roger/IJB/IJB/venv/bin/python /home/roger
```

```
2
```

```
Process finished with exit code 0
```

- Les tuples son immutables. No es poden modificar.

```
myTupla = (1, 2, "pedro", 3, 4, "pedro", 5, 6, 7)
myTupla[3] = "juan"
print(myTupla)
```

```
tupla x
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/tupla.py
Traceback (most recent call last):
  File "/home/roger/IJB/IJB/M4/UF2_PYTHON/TEORIA/tupla.py", line 58, in <module>
    myTupla[3] = "juan"
TypeError: 'tuple' object does not support item assignment
```


- Al ser, la tupla, inmutable, el mètode **append()** no funciona.

```
myTupla = (1,2,"pedro",3)
myTupla.append(3)
print(myTupla)
```

```
tupla x
/home/roger/IJB/IJB/venv/bin/python
Traceback (most recent call last):
  File "/home/roger/IJB/IJB/M4/UF2_
    myTupla.append(3)
AttributeError: 'tuple' object has
Process finished with exit code 1
```

- És una **col·lecció desordenada** sense elements duplicats.
- Suporten operacions matemàtiques com unió, intersecció, diferència i diferència simètrica.
- Els valors no es poden canviar, però es poden borrar i afegir.
- S'inicialitza amb $\Rightarrow$ **mySet = set()**

```
mySet = set()
```

- Per buscar la diferència entre varis sets s'utilitza `-`.

```
x = set('a,b,c,d,e')  
print(x)  
y = set('y,b,z,d,j')  
print(y)  
print(y-x)  
print(x-y)
```

```
set x  
/home/roger/IJB/IJB/venv/bin/python  
{ 'c', 'a', 'b', 'e', ' ', ' ', 'd' }  
{ 'z', 'b', 'y', 'j', ' ', ' ', 'd' }  
{ 'y', 'j', 'z' }  
{ 'a', 'e', 'c' }  
Process finished with exit code 0
```

Interessant que mireu de donar-li al “run” varies vegades per veure com varia la resposta.

- Per unir varis sets s'utilitza `|`.

```
x = set('a,b,c,d,e')  
y = set('y,b,z,d,j')  
print(x | y)
```

set x

```
/home/roger/IJB/IJB/venv/bin/python  
{'a', 'j', 'd', 'z', 'c', 'b', 'y',
```

Process finished with exit code 0

- Per saber els valors repetits en varis sets s'utilitza la intersecció **&**.

```
x = set('a,b,c,d,e')  
y = set('y,b,z,d,j')  
print(x & y)
```

set x

```
/home/roger/IJB/IJB/venv/bin/python  
{'b', 'd', ' ', '}'
```

Process finished with exit code 0

- Per saber els valors no repetits en varis sets s'utilitza la **simetria (XOR)**.

```
x = set('a,b,c,d,e')  
y = set('y,b,z,d,j')  
print(x ^ y)
```

set x

```
/home/roger/IJB/IJB/venv/bin/python  
{'e', 'c', 'y', 'j', 'z', 'a'}
```

Process finished with exit code 0

- Per afegir valors a set s'utilitza **add()**.

```
x = set('a,b,c,d,e')  
x.add('pedro')  
print(x)
```

```
set x  
/home/roger/IJB/IJB/venv/bin/python  
{'d', 'a', 'pedro', 'c', ',', 'e', '  
Process finished with exit code 0
```

Interessant que mireu de donar-li al “run” varies vegades per veure com varia la resposta.

- Per borrar valors a set s'utilitza **remove()**.

```
x = set('a,b,c,d,e')  
x.remove('c')  
print(x)
```

set x

```
/home/roger/IJB/IJB/venv/bin/python  
{ 'd', ' ', 'b', 'a', 'e' }
```

Process finished with exit code 0

- És un set desordenat que conté parells de “key:value”.
- El contingut està indexat per “key”.
- S’inicialitza amb ⇒ **myDict= {}**

```
myDict = {}
```

- Per afegir una key de nom **prova** i el seu valor:

```
myDict = {}  
myDict["prova"] = 18  
print(myDict)
```

dictionary x

```
/home/roger/IJB/IJB/venv/bin/python  
{'prova': 18}
```

Process finished with exit code 0

- Per saber el tamany del diccionari s'utilitza **len()**:

```
myDict = {'prova':18, 'Roger':'Sobrinó', 'Edat': 38}  
print(len(myDict))
```

dictionary x

```
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/  
3
```

Process finished with exit code 0

- Per eliminar l'últim ítem del diccionari s'utilitza **popitem()**.

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}  
print(myDict.popitem())  
print(myDict)
```

dictionaryes x

```
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB,  
('Edat', 38)  
{'prova': 18, 'Roger': 'Sobrino'}
```

Process finished with exit code 0

- Per eliminar contingut amb coincidència de la key:

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}  
del myDict['Edat']  
print(myDict)
```

🐍 dictionaries x

```
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB/  
{'prova': 18, 'Roger': 'Sobrino'}
```

Process finished with exit code 0

- Per saber tots els valors d'un diccionari s'utilitza **values()**.

```
myDict = {'prova':18, 'Roger':'Sobrino', 'Edat': 38}  
print(myDict.values())
```

dict_dictionaries x

```
/home/roger/IJB/IJB/venv/bin/python /home/roger/IJB/IJB  
dict_values([18, 'Sobrino', 38])
```

Process finished with exit code 0

MOLTES GRÀCIES!

